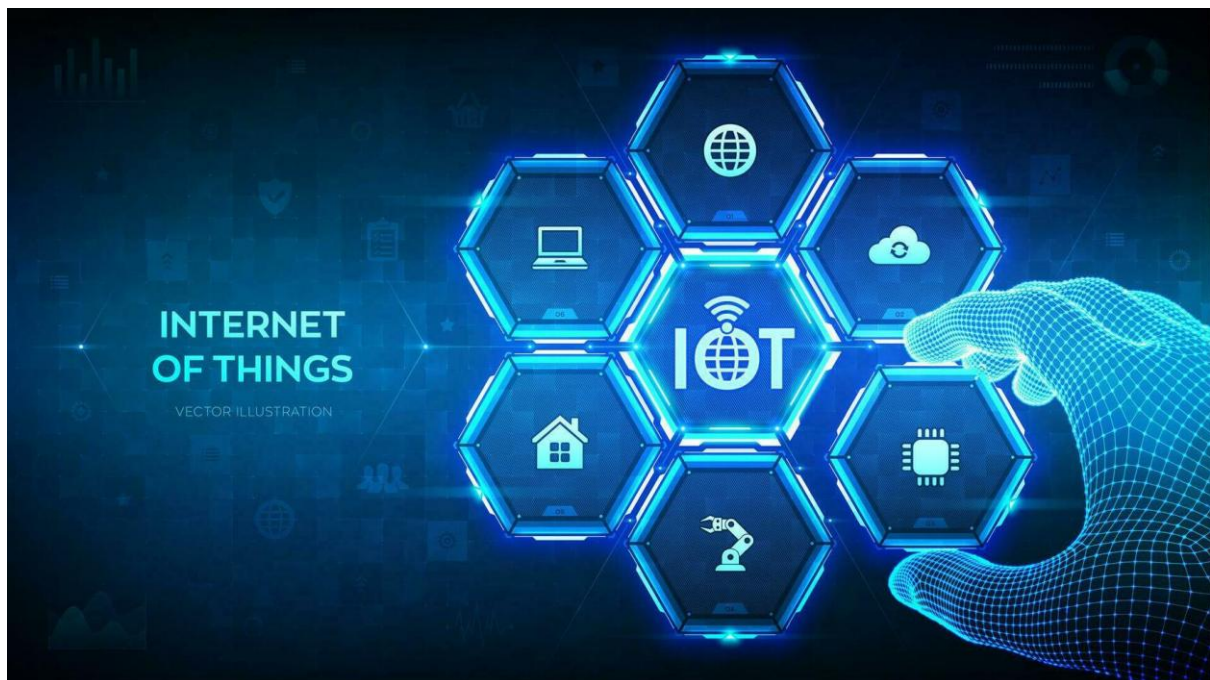


COURSE CODE : CS5202Y25



DOCUMENTATION AND PUBLICATION



Submitted By :
Sparsh Khandelwal (AU23B1021)
Abhay Choudhary (AU23B1003)

Submitted To :
Mr. Satyajit Pangaonkar Sir

Smart Irrigation and Soil Analysis

Project Title: Smart Soil Health and Irrigation System

Subtitle: Real-time IoT monitoring and automated advisory for modern farming.

Date: November 13, 2025

Type of the Project: Internet of Things (IoT) Application

Problem Statement (The Challenge): Traditional farming relies on delayed and static soil health cards, leading to significant inefficiencies in resource management. Farmers lack real-time data on critical soil parameters like moisture, pH, and temperature. This results in inefficient, manual irrigation practices, unnecessary water waste, and suboptimal crop yields due to missed opportunities for timely nutritional correction. There is a critical need for an accessible, automated system that provides immediate soil analysis and smart advisory.

The Solution Highlight: The Smart Irrigation and Soil Analysis system solves this challenge by integrating **ESP32-based IoT sensors** (pH, moisture, temperature) to capture **real-time soil data**. This data is securely stored on **Firestore** and visualized on a **Blynk dashboard** and web interface. The system automates irrigation when moisture levels drop below a crop-specific threshold and provides instant recommendations for pH correction, crop suitability, and disease prevention, maximizing water efficiency and crop health.

Abstract

This report details the design and implementation of the **Smart Irrigation and Soil Analysis** system, an Internet of Things (IoT) application designed to modernize traditional farming. The project addresses the critical gap in real-time agricultural data by deploying an ESP32 microcontroller integrated with soil moisture, pH, and temperature sensors. By leveraging cloud platforms (Firestore/Blynk), the system provides farmers with an accessible, real-time dashboard for soil health monitoring. Key functionality includes **automated irrigation control** based on pre-set crop

thresholds, instantaneous pH correction advisories, and comprehensive crop management guides (info, diseases, harvesting). This system aims to significantly reduce water consumption, improve soil fertility management, and ultimately increase overall agricultural productivity.

Problem Statement and Objective

Problem Statement

Modern agriculture suffers from poor data accessibility, leading to inefficient resource use. The reliance on delayed, periodic soil reports and manual decision-making results in over-irrigation, fertilizer misuse, and preventable crop loss. A solution is required to bridge the gap between sensor data and actionable advice for farmers.

Project Objectives

- **Monitor Soil Health in Real-time:** Continuously track pH, moisture, and temperature using integrated sensors.
- **Automate Irrigation:** Automatically activate the water pump/relay when soil moisture drops below the threshold optimal for the selected crop.
- **Provide Smart Advisory:** Recommend pH correction methods and suggest suitable crops based on the soil fertility index.
- **User Accessibility:** Deliver data, alerts, and advice through an intuitive, mobile/web-accessible dashboard.
- **Digital Records:** Provide a digital Soil Health Card for long-term tracking and historical analysis.

Introduction

The global demand for food security, combined with increasing concerns over water scarcity, necessitates the adoption of intelligent farming practices. IoT technology offers a powerful solution by enabling precision

agriculture. The Smart Irrigation and Soil Analysis project introduces an end-to-end solution that integrates hardware (sensors, ESP32), connectivity (WiFi), and a user interface (Cloud/Blynk) to provide a centralized, autonomous system for farm management. The core value lies in converting raw environmental data into immediate, actionable intelligence for the farmer.

Ideation

The ideation phase focused on maximizing impact and accessibility for the farmer. Initial brainstorming generated ideas beyond basic sensing, including AI-driven fertility forecasts, integration with local weather APIs for predictive irrigation, and multi-lingual voice-based UIs for accessibility. The final system architecture was chosen to deliver the most critical features immediately: **real-time sensing** and **automated resource control**. The use of a simple, secure cloud platform like Blynk/Firebase ensures the data is always available, a significant improvement over static traditional methods.

Design and Methodology

System Architecture

The system follows a standard IoT architecture:

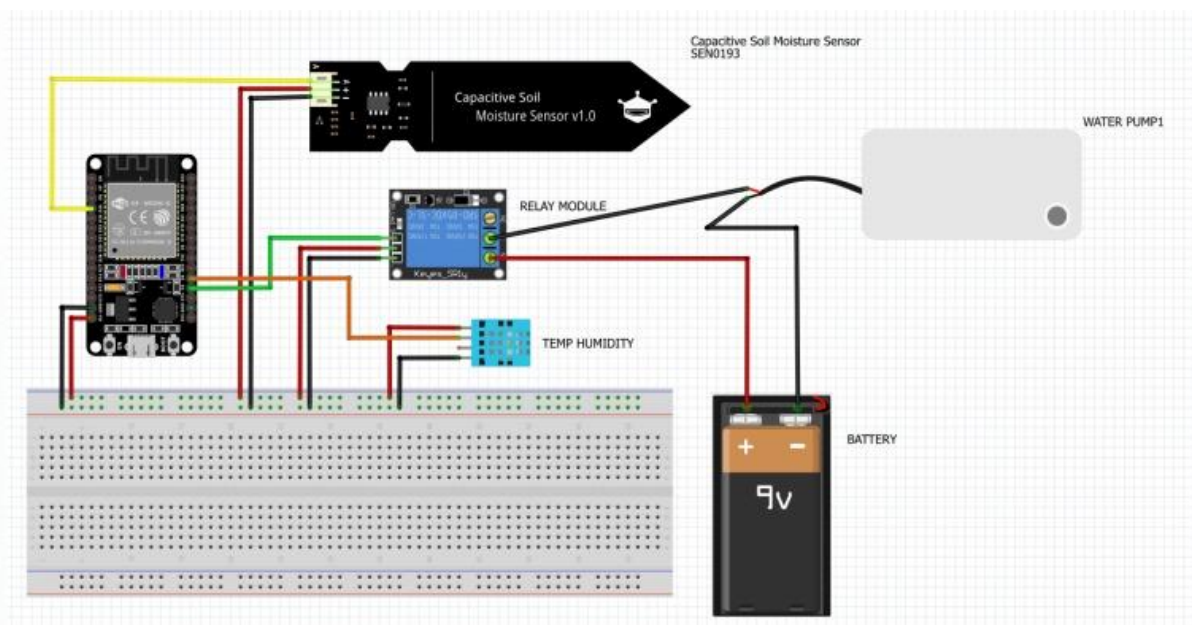
1. **Sensing Layer (Edge):** Soil moisture, pH, and temperature sensors collect data.
2. **Processing & Connectivity Layer:** An **ESP32** microcontroller reads sensor data via its Analog-to-Digital Converter (ADC), processes it, and transmits it to the cloud via Wi-Fi. It also receives control commands (ON/OFF) for the relay.
3. **Cloud Layer: Firebase** (or a similar cloud service) provides secure storage for sensor data and crop thresholds. **Blynk** (or a custom web UI) is used for real-time visualization and control.

4. **Application Layer:** The mobile/web dashboard presents live data, crop information, and allows for manual override of the water pump.

Circuit Diagram and Components

The hardware setup is designed for robustness and low power consumption.

Component	Function
ESP32	Microcontroller, Wi-Fi connectivity, and sensor data processing.
Capacitive Soil Moisture Sensor	Measures soil moisture content (analog output).
Temperature/Humidity Sensor	Measures ambient and soil temperature (e.g., DHT11/DS18B20).
pH Sensor	Measures soil acidity/alkalinity.
Relay Module	Acts as an electronic switch to safely control the high-power water pump.
Water Pump	Physical mechanism for irrigation.



Implementation

The implementation involves two main parts: the firmware (C++/Arduino for ESP32) and the web/cloud interface.

Key Functionality and Code Snippets

1. Real-Time Data Acquisition (Firmware)

The ESP32 reads the analog values from the moisture sensor and converts them into a percentage based on calibrated dry and wet readings.

Explanation: The `readMoisture()` function is executed periodically. It uses `analogRead()` to get the raw sensor data (e.g., 0-4095) and maps it to a calibrated range (e.g., 0-100%) for easy interpretation.

Code Snippet (Conceptual C++ for ESP32):

```
poe_final_code.ino
1  #include <WiFi.h>
2  #include <WebServer.h>
3  #include <DHT.h> // Include DHT sensor library
4
5  // ----- WiFi Credentials -----
6  const char* ssid      = "KALIDAS";
7  const char* password = "Avantika!";
8
9  // ----- Login Credentials -----
10 const char* loginUser = "admin";
11 const char* loginPass = "1234";
12
13 // ----- Pin Config -----
14 #define SOIL_MOISTURE_PIN 34
15 #define PUMP_PIN 2
16 #define DHT_PIN 4 // GPIO pin for DHT11
17
18 // ----- DHT Sensor Setup -----
19 #define DHT_TYPE DHT11
20 DHT dht(DHT_PIN, DHT_TYPE);
21
22 bool isPumpOn = false;
23 bool isLoggedIn = false;
24 bool isCropSelected = false;
25 String selectedCrop = "None";
26
27 WebServer server(80);
28
29 // ----- Simulated Sensor Data (keeping for internal logic but not displaying) -----
30 int simulatedPH = 6; // placeholder
31 int simulatedN = 40;
32 int simulatedP = 20;
33 int simulatedK = 30;
--
```

poe_final_code.ino

```
127
128 // ----- Sensor Functions -----
129 int readSoilMoisture() {
130     int rawValue = analogRead(SOIL_MOISTURE_PIN);
131     int percentage = map(rawValue, 3500, 4095, 100, 0);
132     return constrain(percentage, 0, 100);
133 }
134
135 float readTemperature() {
136     return dht.readTemperature(); // Read temperature in Celsius
137 }
138
139 float readHumidity() {
140     return dht.readHumidity(); // Read humidity percentage
141 }
142
143 // ----- Get Crop Thresholds -----
144 CropThreshold getCropThresholds(String cropName) {
145     if (cropName == "Wheat") return wheat;
146     else if (cropName == "Rice") return rice;
147     else if (cropName == "Tomato") return tomato;
148     else if (cropName == "Potato") return potato;
149     else if (cropName == "Onion") return onion;
150     // Return a default/safe threshold if not found
151     return {40, 80, 6.0, 7.0, 30, 20, 20, 15, 30, 40, 80}; // Generic safe values
152 }
153
```

poe_final_code.ino

```
154 // ----- Soil and Environment Health Check -----
155 String checkSoilHealth(int soil, float ph, int N, int P, int K, float temp, float humidity) {
156     CropThreshold th = getCropThresholds(selectedCrop);
157
158     if (selectedCrop == "None") return "Please select a crop for tailored advice.";
159
160     String msg = "";
161
162     // Soil moisture check
163     if (soil < th.minMoisture) msg += "Moisture too low<br>";
164     else if (soil > th.maxMoisture) msg += "Moisture too high<br>";
165
166     // pH check
167     if (ph < th.minPH || ph > th.maxPH) msg += "pH not suitable<br>";
168
169     // Nutrient checks
170     if (N < th.minN) msg += "Low Nitrogen<br>";
171     if (P < th.minP) msg += "Low Phosphorus<br>";
172     if (K < th.minK) msg += "Low Potassium<br>";
173
174     // Temperature check
175     if (temp < th.minTemp) msg += "Temperature too low<br>";
176     else if (temp > th.maxTemp) msg += "Temperature too high<br>";
177
178     // Humidity check
179     if (humidity < th.minHumidity) msg += "Humidity too low<br>";
180     else if (humidity > th.maxHumidity) msg += "Humidity too high<br>";
181
182     if (msg == "") msg = "Environment is suitable for " + selectedCrop;
183     return msg;
184 }
```

Testing and Deployment

Testing

The system underwent rigorous functional testing (as per the testing chart provided in the original document) to validate connectivity, sensor accuracy, and the automation logic.

Key Test Cases:

- **Moisture Check:** Input: Moisture < crop threshold (e.g., 40% for a 50% min). Expected Output: Pump ON. **Result: PASSED.**
- **Manual Pump Control:** Input: Farmer clicks ON button. Expected Output: Pump starts for the selected area. **Result: PASSED.**
- **Farmer Login:** Successful authentication to ensure data security.

Deployment

The ESP32 is deployed in the field, powered by a battery (optional/solar). The device connects to the farmer's Wi-Fi network. The cloud dashboard (Blynk/Web UI) is accessible via any modern browser or mobile application, allowing for remote monitoring and management from anywhere.

Result and Conclusion

Result

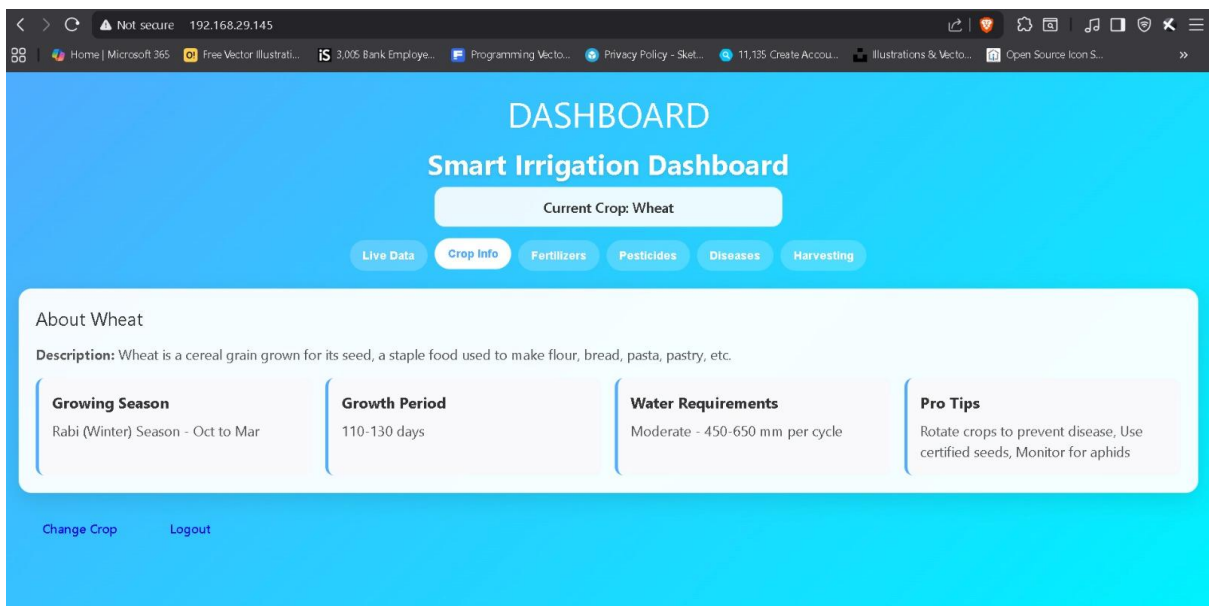
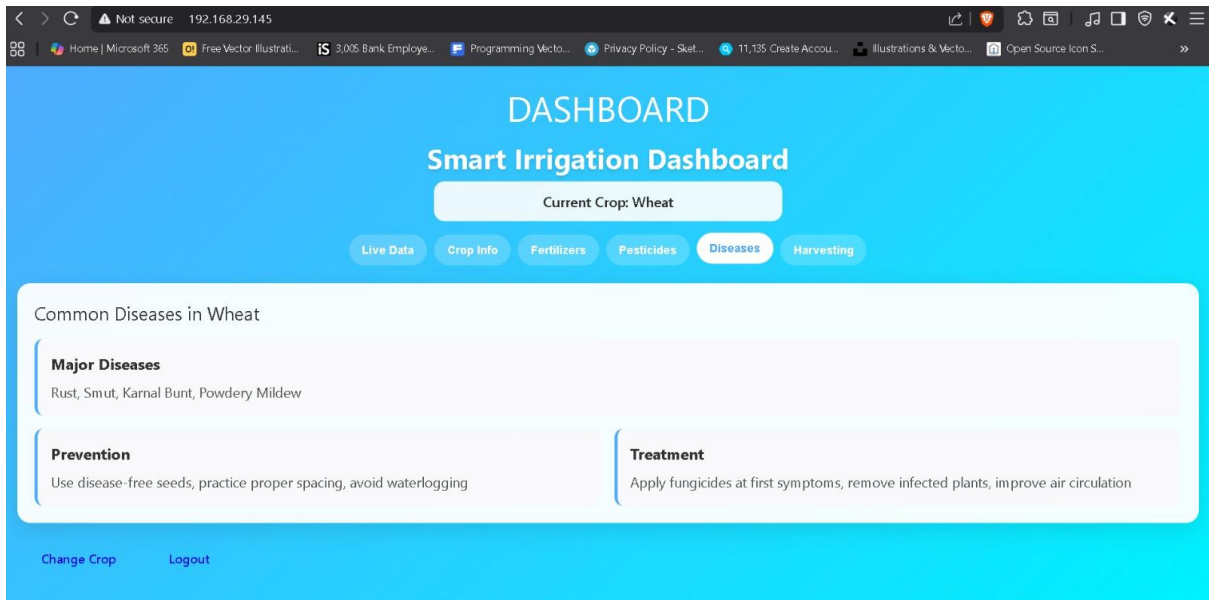
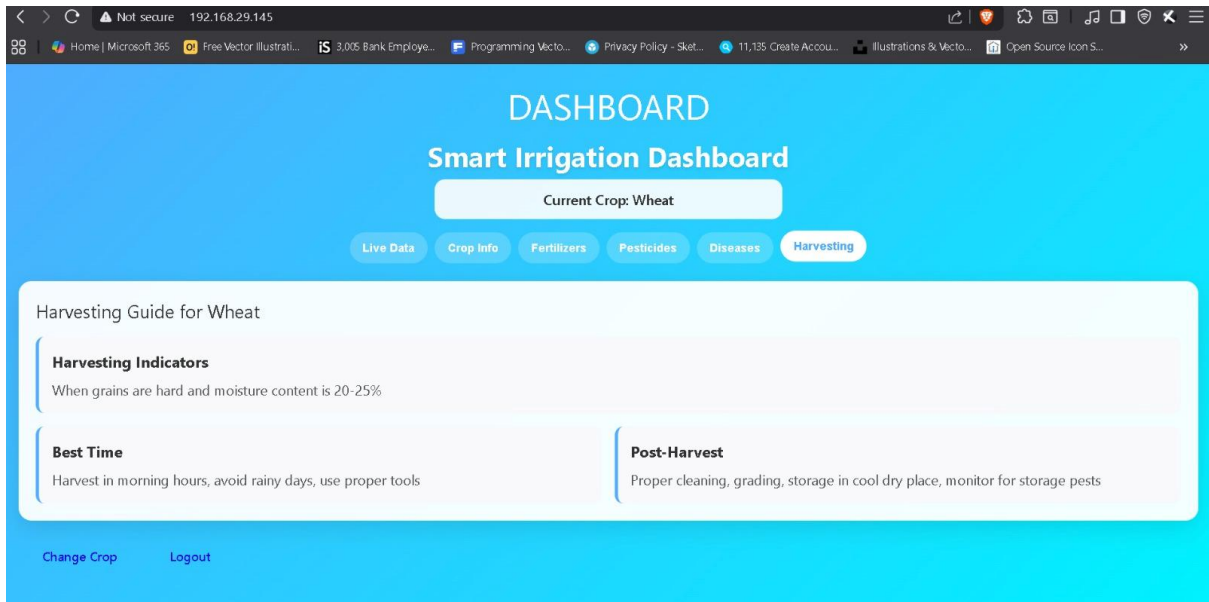
The Smart Irrigation and Soil Analysis system successfully delivers its core objectives. It provides a reliable platform for real-time soil parameter monitoring, achieving water use efficiency through automated, data-driven irrigation. The integration of crop-specific data, disease information, and harvesting guides elevates the solution from a simple monitoring tool to a comprehensive farm advisory service.

Snapshot: Dashboard - Crop Information

The screenshot shows a web browser window with the URL 192.168.29.145. The page has a purple gradient background. In the center is a white card titled "PLANT Smart Irrigation System". It contains a login form with fields for "Username" and "Password", each with a placeholder "Enter your username" and "Enter your password" respectively. Below the fields is a blue button labeled "Login to Dashboard". At the bottom of the card, it says "Secure Agricultural Monitoring".

The screenshot shows a web browser window with the URL 192.168.29.145. The page has a pink gradient background. In the center is a white card titled "CROP Select Your Crop". Below the title is the text "Choose the crop you're growing for customized monitoring". There is a dropdown menu with the placeholder "Select a crop...". The dropdown is open, showing a list of crops: "Wheat", "Rice", "Tomato", "Potato", and "Onion".

The screenshot shows a web browser window with the URL 192.168.29.145. The page has a blue gradient background. At the top, it says "DASHBOARD Smart Irrigation Dashboard". Below this is a button labeled "Current Crop: Wheat". There are six tabs: "Live Data", "Crop Info", "Fertilizers", "Pesticides", "Diseases", and "Harvesting". The "Live Data" tab is active. Below the tabs are three white cards showing sensor data: "Soil Moisture" with a value of "--%" and an optimal range of "Optimal: 50-70%", "Temperature" with a value of "--Â°C" and an optimal range of "Optimal: 15-25Â°C", and "Humidity" with a value of "--%" and an optimal range of "Optimal: 40-70%". At the bottom is a blue card for the "Water Pump" with a "Status: --" and a "Toggle Pump" button. Below the button, it says "Auto-mode: Moisture < 40%".



Conclusion

The project demonstrates the immense potential of IoT in transforming agriculture. By offering an accessible, automated, and intelligent system, it directly tackles the problems of delayed information and inefficient resource allocation. Future enhancements could involve integrating weather APIs for predictive irrigation and implementing multilingual voice assistants to further enhance farmer accessibility.

A hand holding a smartphone is shown in a soft-focus background. Overlaid on the image is a green graphic consisting of a circular line with two leaves at its ends. The words "Thank you" are written in a bold, black, sans-serif font in the center of the circle.

**Thank
you**